Attendance Module – Functional Overview

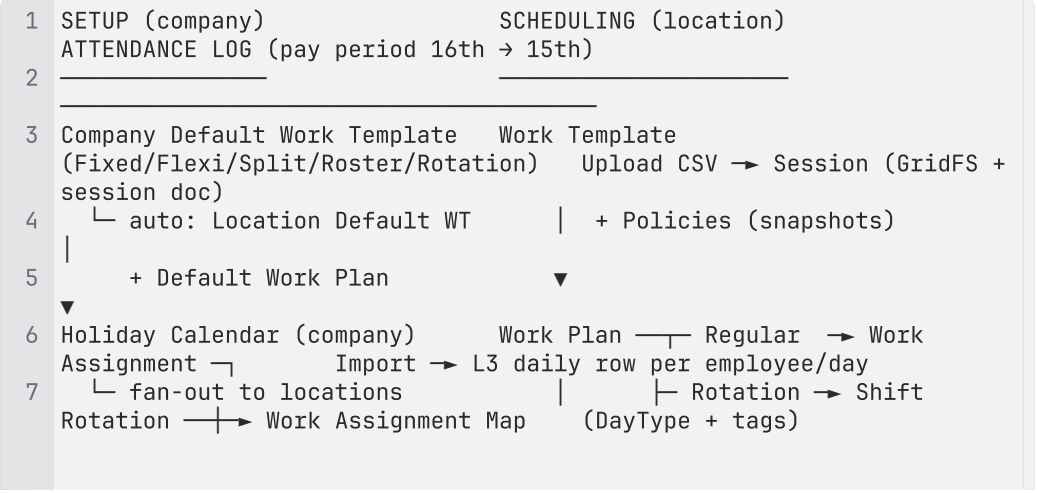
> This page explains how the Attendance module works from start to finish: how a company sets up its schedule, holidays and policies; how employees get a shift (regular, rotation or roster); and how attendance files become daily (L3) and monthly (L4) records. Each section gives the flow, the main rules and one sample of the stored MongoDB document. The samples are cut down, and their ids and names are made up. For field-level detail, see the Attendance Service Walkthrough.

Table Of Contents

1. End-to-end flow
2. Company setup
   - 2.1 Company schedule (Company Default Work Template)
   - 2.2 Holiday calendar
   - 2.3 Policies
   - 2.4 Attendance configuration
3. Work templates
4. Work plans and regular assignment
   - 4.1 Work plan
   - 4.2 Work assignment and work assignment map
5. Shift rotation
6. Roster
7. Attendance import (file to L3)
8. L4 processing, edits and publish
   - 8.1 Finalise (L3 to L4)
   - 8.2 Editing L3 days
9. Points to note

1. End-to-end flow

Scheduling and attendance capture are two separate tracks. Scheduling decides **what an employee is expected to work** on each date. The attendance log records **what they actually worked**, imported from a file, and turns it into daily and monthly figures.



```
Policies (Grace/OT/Break)            |      └─ Roster   ─► Roster Board
└──┘   (employee × date range)    |
Attendance Global Config             |
▼
  (day-offs, file level)             └─ "Which shift applies to
employee X on date D?"  Finalise ─► L4 monthly per employee

Edit L3 ─► re-process ─► Publish
```

Layer	What it answers	Collection
Work Template	The weekly time pattern (days and hours)	`ERP.Attendance.WorkTemplates`
Policy	Grace, overtime and break rules	`ERP.Attendance.Policies`
Work Plan	Template + policies, packaged for one location	`ERP.Attendance.WorkPlans`
Work Assignment / Map	Which employee follows which plan, and from when to when	`ERP.Attendance.WorkAssignments`, `ERP.Attendance.WorkAssignmentMaps`
Rotation / Roster	Plans that change in a cycle (rotation) or are set day by day by a planner (roster)	`ERP.Attendance.ShiftRotationBoard`, `ERP.Attendance.RosterBoards`, … `RosterSchedulePeriods`, … `EmployeeRosterAssignments`
Holiday Calendar	Public holidays per company or location, per year	`ERP.Attendance.HolidayCalendars`
Attendance log	Imported attendance, then daily (L3) and monthly (L4) results	`ERP.Attendance.Log.*`

Key principle: copies, not links. Each layer keeps a copy (snapshot) of the layer below it. A plan holds a copy of its template and policies, and an assignment map holds a copy of the

plan and the employee. If a master record changes later, existing copies stay the same until they are refreshed on purpose.

2. Company setup

2.1 Company schedule (Company Default Work Template)

- Each company has one active **Company Default Work Template (CDWT)**. It sets the standard working week and can be Fixed, Flexi or Split.
- When the CDWT is created, every company location automatically gets a linked **Location Default** template and a **Default Work Plan** (short code `LOCWOPL`).
- Editing the CDWT creates a new version (1.0 → 2.0) and archives the old one. The locations pick up the change only when an admin runs **apply-sync**.

2.2 Holiday calendar

- A calendar is stored per company or location and per year. The company default calendar is created first, and a copy is sent to every location. Those copies are marked *inherited* and are read-only.
- Edits to the company calendar are staged until someone **publishes** them. Publishing overwrites every location calendar that is still inherited. A location that has been **customised** keeps its own list and is not overwritten.
- Only the current year and the next year can be edited. Holidays are stored as date ranges.
- Holidays affect how each day is classified (Holiday, or Holiday on Day Off) in roster and rotation schedules, the leave day-classification API and the attendance log. An employee's location, and so their holiday calendar, comes from their work assignment map.

Mongo example – HolidayCalendar (company default)

```
{
  _id: UUID("5555…"), TenantId: UUID("aaaa…"),
  CompanyId: UUID("c000…"), CompanyName: { en: "Demo Company" },
  LocationId: UUID("00000000-0000-0000-0000-000000000000"),   // company default
  ShortCode: "HC-2027", Name: { en: "Company Holidays 2027" },
  CalendarType: 1,            // 1 CompanyDefault, 2 LocationDefault
  IsInherited: false, IsPublished: false, IsActive: true,
  LastPublishedOn: ISODate("2026-09-05T09:00:00Z"),
  YearlyCalendar: {
    Year: 2027,
    HolidayLists: [
      { _id: UUID("6666…01"), HolidayName: { en: "New Year Holiday" },
        HolidayType: { _id: UUID("7000…01"), Name: { en: "Public Holiday" } },
        StartDate: { Year: 2027, Month: 1, Day: 1 }, EndDate: { Year: 2027, Month: 1, Day: 1 } }
    ]
  }
}
```

2.3 Policies

- There are three policy types, all set at company level: **Grace Period** (late check-in and early check-out allowance), **Overtime** and **Break**. A company can have at most 30 policies. Deleting a policy is a soft delete.
- Policies are attached to a Work Plan. The plan stores a copy of each one, and that copy then flows into every assignment map.

Mongo example – Grace Period policy

```
{
  _id: UUID("1111…"), _t: ["PolicyBase", "GracePeriodPolicy"],
  CompanyId: UUID("c000…"), Name: { en: "Standard Grace 15m" },
IsActive: true,
  Type: 3,                          // 1 Break, 2 Overtime, 3
GracePeriod
  LateCheckInGraceMinutes: 15, EarlyCheckOutGraceMinutes: 10,
  DeviationThreshold: 3, DeviationPeriod: 3, DeductLeaveFrom: 1,
DeductionDays: 0.5
}
```

2.4 Attendance configuration

- **Attendance Global Config** (`ERP.Attendance.Log.AttGlobalConfig`) has one document per company. It holds the company **day-offs** (Saturday and Sunday by default), the allowed upload file level (L3) and the expected file headers.
- **Employee exemptions** (`ERP.Attendance.Configs`) list employees who are exempt from attendance for a date range.

3. Work templates

A Work Template is a weekly pattern with one row per weekday, and it is the base of every schedule. Each template has a short code and a timezone, and the system works out the total working days and hours.

Type	TemplateType	Scope	Behaviour
Fixed	1	Company or location	One start and end time per day
Flexi	2	Company or location	A required duration only (daily or weekly), with no clock times
Split	3	Company or location	Several time blocks in one day
Roster	4	Location only	One shift with a colour. An overnight shift is detected automatically. Used only by roster boards.

Rotation	5	Location only	Shaped like Fixed, with all 7 days required. Used only by rotation plans and never a default.

Rules: a regular template (Fixed, Flexi or Split) can only change to another regular type. Roster and Rotation templates keep their type for life. Once a plan uses a template, the template is locked for editing and can only be archived.

Mongo example – Fixed work template

```
{
  _id: UUID("a100…02"), _t: ["WorkTemplate", "FixedWorkTemplate"],
  CompanyId: UUID("2222…"), LocationId: UUID("00000000-…"),   // empty = company level
  WorkTemplateName: { en: "Company Standard" }, ShortCode: "COSTD01",
  TemplateType: 1, Status: 1, Version: "2.0", IsDefault: true, TimeZoneId: "Asia/Riyadh",
  TotalWorkDurationInMinutes: 2400, TotalWorkingDays: 5, TotalDaysOff: 2,
  HasPendingLocSync: true,
  FixedTimeSlots: [
    { DayEnum: 1, IsWorkingDay: true,  StartTime: "09:00:00", EndTime: "17:00:00", WorkDurationInMinutes: 480 },
    // … DayEnum 2-5 the same …
    { DayEnum: 6, IsWorkingDay: false, StartTime: "00:00:00", EndTime: "00:00:00", WorkDurationInMinutes: 0 },
    { DayEnum: 7, IsWorkingDay: false, StartTime: "00:00:00", EndTime: "00:00:00", WorkDurationInMinutes: 0 }
  ]
}
```

4. Work plans and regular assignment

4.1 Work plan

A Work Plan belongs to one location. It holds **one template copy** and **zero or more policy copies**, and it is what employees are assigned to. There are three kinds:

- **Regular** (`PlanType 1`): managed through the Work Plans screen. Each location has one default plan.
- **Roster** (`PlanType 2`): a shift on a roster board. It is created together with its own Roster template and carries `BoardId` .
- **Rotation** (`PlanType 3`): one plan slot (P1 to P4) of a shift rotation. It carries `RotationId` and `PlanIndex` .

Mongo example – Regular work plan

```
{
  _id: UUID("1111…"), _t: ["WorkPlan", "RegularWorkPlan"], PlanType: 1,
  WorkPlanName: { en: "General Shift" }, ShortCode: "GENSHFT",
  CompanyId: UUID("…c1"), LocationId: UUID("…e1"), LocationName: { en: "Head Office" },
  IsDefault: false, IsDeleted: false,
  IsGracePeriodPolicyEnabled: true, IsOvertimePolicyEnabled: false, IsBreakPolicyEnabled: false,
```

```
  Policies: [ { _t: ["EmbeddedPolicy", "GracePeriodPolicy"], _id:
UUID("3333…"), Type: 3,
                Name: { en: "Standard Grace" },
LateCheckInGraceMinutes: 10, EarlyCheckOutGraceMinutes: 10 } ],
  WorkTemplate: { _t: ["EmbeddedWorkTemplate",
"FixedWorkTemplate"], _id: UUID("4444…"),
                 WorkTemplateName: { en: "Office 9-6" },
TemplateType: 1, TimeZoneId: "Asia/Dubai",
                 FixedTimeSlots: [ { DayEnum: 1, IsWorkingDay:
true, StartTime: "09:00:00", EndTime: "18:00:00" } /* … */ ] }
}
```

4.2 Work assignment and work assignment map

1. The admin chooses a Regular plan, the employees and a date range. The end date can be left open.
2. **Check conflicts** (preview only): the system lists employees who already have an active assignment at the same location in that window.
3. **Create**: the system writes one **Work Assignment** header (plan, location and dates, with no employee list) and one **Work Assignment Map** for each employee without a conflict.
4. **Adjust** (optional): for the employees who had conflicts, the old regular maps are shortened, split or deactivated, and new maps are created. An overlap with a roster or rotation day is never adjusted (error 1408).
5. Later actions: edit the period, remove an employee (`IsActive=false`), or move an employee to another plan, board or rotation. A move takes effect from tomorrow by default.

Rule: an employee can have only one active Regular assignment per date at a location. An open end date is stored as `9999-12-31` and returned as `null` by the API.

The Work Assignment Map is the per-employee answer. One collection holds all three kinds:

Map kind	Assignment Type	One document per	Created by
Regular	1	Employee × period	Work assignment create, adjust or move
Roster	2	Employee × one date	Roster schedule assign
Rotation	3	Employee × enrolment × step window	Rotation enrolment (step generator)

Which shift applies to employee X on date D? The system takes the active maps for X that cover D and uses the one with the latest start date. That map's `WorkPlanData` provides the

template and policies, and its `LocationId` and `HolidayCalendarId` provide the holidays.

Mongo example – Regular work assignment map

```
{
  _id: UUID("b2b2…01"), _t: ["WorkAssignmentMap", "RegularWorkAssignmentMap"], AssignmentType: 1,
  CompanyId: UUID("…c1"), LocationId: UUID("…e1"), LocationName: { en: "Head Office" },
  StartDate: ISODate("2026-10-01"), EndDate: ISODate("9999-12-31"), // open-ended
  Status: 1, IsActive: true,
  EmployeeId: UUID("e1e1…01"), EmployeeCode: "EMP0001", EmployeeName: { en: "Test Employee One" },
  Department: { EntityId: UUID("d1d1…"), Name: { en: "Operations" }, ShortCode: "OPS" },
  ReportingTo: { EntityId: UUID("e8e8…"), Name: { en: "Test Manager" }, EmployeeCode: "EMP0008" },
  WorkAssignmentId: UUID("a1a1…01"),          // header (Regular only)
  HolidayCalendarId: UUID("f1f1…01"),
  WorkPlanData: {
    WorkPlanId: UUID("1111…"), WorkPlanName: { en: "General Shift" },
    Policies: [ /* copy of the plan's policies */ ],
    WorkTemplate: { _t: ["EmbeddedWorkTemplate", "FixedWorkTemplate"], TemplateType: 1, FixedTimeSlots: [ /* … */ ] }
  }
}
```

5. Shift rotation

A rotation belongs to one location. It moves teams through **2 to 4 rotation plans** (P1 to P4) in a fixed cycle. A cycle has 2 to 8 steps, each step lasts 1 to 13 weeks, and the whole cycle lasts 2 to 52 weeks.

1. Create one **Rotation work template** for each shift (for example Day, Evening and Night).
2. Create the **rotation** with its name, pattern type (Alternating or Uneven), weeks per step and plans. The system creates one Rotation Work Plan per slot, builds the teams, and gives each team the same pattern starting at a different step.
3. **Enrol employees** into a team with a start date. Overlapping regular maps at the location are ended. An employee who is already on a board or another rotation is refused (2016).
4. The generator writes **one assignment map per step window**, covering up to 104 weeks ahead. *Generate-schedule* can top this up and is safe to run again.

There is no rotation start date. Each employee's cycle starts on their own enrolment start date. Example: a member of Team B (pattern P2 → P3 → P1, 2 weeks each) who joins on 2026-10-04 works P2 from 10-04 to 10-17, P3 from 10-18 to 10-31, P1 from 11-01 to 11-14, and then the cycle repeats. The daily hours come from the step's template row for that weekday. The structure (plans, pattern, teams) can only be changed when no member is current or upcoming (2018).

Mongo example – ShiftRotation header (trimmed)

```
{
  _id: UUID("3333…"), RotationName: { en: "Plant Ops 3-Shift" },
  CompanyId: UUID("1111…"), LocationId: UUID("2222…"),
  PatternType: 1, WeeksPerStep: 2,                  // Alternating
  Plans: [ { PlanIndex: 1, WorkPlanId: UUID("4444…01"),
WorkPlanName: { en: "Day Plan" },
             Template: { _t: ["EmbeddedWorkTemplate",
"RotationWorkTemplate"], TemplateType: 5, RotationTimeSlots: [ /* 7
days */ ] },
             Policies: [] } /* P2 Evening, P3 Night */ ],
  Teams: [ { TeamId: UUID("6666…02"), TeamName: { en: "Team B" },
DisplayOrder: 2,
             RotationPattern: [ { SequenceOrder: 1, PlanIndex: 2,
DurationWeeks: 2 },
                                { SequenceOrder: 2, PlanIndex: 3,
DurationWeeks: 2 },
                                { SequenceOrder: 3, PlanIndex: 1,
DurationWeeks: 2 } ] } /* Team A, Team C */ ],
  Members: [ { Id: UUID("7777…01"),               // MembershipId
               TeamId: UUID("6666…02"), EmployeeId: UUID("8888…
01"), EmployeeCode: "EMP-0001",
               PeriodStartDate: ISODate("2026-10-04"),
PeriodEndDate: ISODate("9999-12-31"),
               GeneratedThrough: ISODate("2028-09-30") } ],
  IsDeleted: false
}
```

Mongo example – Rotation step (work assignment map)

```
{
  _t: ["WorkAssignmentMap", "RotationWorkAssignmentMap"],
AssignmentType: 3,
  EmployeeId: UUID("e3e3…03"), EmployeeCode: "EMP0003",
  StartDate: ISODate("2026-10-04"), EndDate: ISODate("2026-10-17"),
// one step window
  WorkAssignmentId: null,
  WorkPlanData: { WorkPlanId: UUID("8888…"), WorkPlanName: { en:
"Night Rotation Plan" },
                  WorkTemplate: { _t: ["EmbeddedWorkTemplate",
"RotationWorkTemplate"], TemplateType: 5,
                                  RotationTimeSlots: [ { DayEnum:
1, IsWorkingDay: true, StartTime: "22:00:00", EndTime: "06:00:00" }
/* … */ ] } },
  RotationAssignment: { RotationId: UUID("aaaa…"), TeamId:
UUID("abab…01"), TeamName: { en: "Team A" },
                        MembershipId: UUID("acac…01"), PlanIndex:
2, SequenceOrder: 2, CycleNumber: 1, IsPartial: false }
}
```

6. Roster

A roster suits teams whose shifts a planner sets **day by day**. The flow is: board, then members, then shifts, then the daily schedule, then publish.

1. **Roster board**: one location, a supervisor, a publishing frequency (Weekly, Bi-weekly or Monthly) and optional **areas** (work zones).
2. **Members**: employees are added as membership rows, each with its own start and end dates. An employee can leave and rejoin, but their rows cannot overlap. Conflicts are checked first.
3. **Shifts**: each shift is a Roster Work Plan paired with its own Roster template (one colour and fixed times), and it belongs to exactly one board. The board keeps a display copy of its

shifts.

4. **Assign**: the planner fills the grid (employee × date) with a shift or a Day Off. Each filled cell is stored as **one roster map document** for that employee and date. A dateless **EmployeeRosterAssignment** row records that the employee works that shift on that board. Copy-previous and clear are also available.
5. **Publish** one period at a time. Publishing creates or updates a **RosterSchedulePeriod** row (Draft → Published, version + 1). Any change inside a published period marks it as needing a republish. A period that has ended is frozen (1909).

Notes: weeks start on Sunday. The status shown on screen (Not Planned, Pending, Published, Completed) is worked out when the page loads. Leave and holidays are not stored as roster days; they are shown over the schedule when it is read. Day documents can be read as soon as they are assigned, because publishing changes only the period's status.

Mongo example – RosterBoard (trimmed)

```
{
  _id: UUID("b000…01"), BoardName: { en: "Line A Board" },
  CompanyId: UUID("c000…"), LocationId: UUID("1000…"),
  Supervisor: { EntityId: UUID("e000…99"), Name: { en: "Sample Supervisor" }, EmployeeCode: "EMP0099" },
  PublishingFrequency: 1,                          // Weekly
  HasAreas: true,
  Areas: [ { AreaId: UUID("a000…01"), AreaName: { en: "Area One" }, Order: 0 } ],
  Employees: [ { _id: UUID("d000…01"),             // MembershipId
                 EmployeeId: UUID("e000…01"), EmployeeCode: "EMP0001", AreaId: UUID("a000…01"),
                 PeriodStartDate: ISODate("2026-09-27"), PeriodEndDate: ISODate("9999-12-31") } ],
  WorkPlans: [ { WorkPlanId: UUID("f000…01"), WorkPlanName: { en: "Morning" }, ShortCode: "MORN001",
                 Template: { _t: ["EmbeddedWorkTemplate", "RosterWorkTemplate"], TemplateType: 4, Colour: "#12B886",
                             RosterTimeSlots: [ { DayEnum: 2, StartTime: "08:00:00", EndTime: "16:00:00", CrossesMidnight: false } ] } } ],
  IsDeleted: false
}
```

Mongo example – Roster day (work assignment map) and RosterSchedulePeriod

```
// ERP.Attendance.WorkAssignmentMaps - one employee, one date
{
  _t: ["WorkAssignmentMap", "RosterWorkAssignmentMap"], AssignmentType: 2,
  EmployeeId: UUID("e2e2…02"), StartDate: ISODate("2026-10-05"), EndDate: ISODate("2026-10-05"),
  WorkAssignmentId: null,
  WorkPlanData: { WorkPlanId: UUID("5555…"), WorkPlanName: { en: "Morning" },   // Day Off = empty Guid + "Day Off"
                  WorkTemplate: { _t: ["EmbeddedWorkTemplate", "RosterDayTemplate"], TemplateType: 4, Colour: "#12B886",
                                  FixedTimeSlots: { DayEnum: 2, IsWorkingDay: true, StartTime: "06:00:00", EndTime: "14:00:00" } } },
  RosterAssignment: { RosterBoardId: UUID("7777…"), MembershipId: UUID("c4c4…01"),
                      RosterAssignmentId: UUID("c5c5…01"), IsAssignedArea: true,
```

```
                        Area: { AreaId: UUID("c6c6…01"), AreaName: { en: "Loading Bay" } } }
}
// ERP.Attendance.RosterSchedulePeriods - publish state per (board, period)
{
  BoardId: UUID("b000…01"), Frequency: 1,
  PeriodStart: ISODate("2026-09-27"), PeriodEnd: ISODate("2026-10-03"), PeriodLabel: "Week 39",
  Status: 2, Version: 1,                          // 1 Draft, 2 Published
  PublishedAt: ISODate("2026-09-25T09:00:00Z"), RepublishRequired: false
}
```

7. Attendance import (file to L3)

Attendance enters the module only through a **CSV upload**, which is processed at level **L3** (one row per employee per day). A **session** covers one pay period from the 16th to the 15th (for example `Mar16_2026_to_Apr15_2026`). There is one session per company, period, location and pay group.

1. **Upload**: the system checks the file level and headers, and rejects a file whose dates fall outside the period (1801/1802). The file is stored in GridFS and added to the session (state Draft).
2. **Import**: each row is matched to an active **Employee Enrollment** in the pay group using `EmployeeCode`.
3. **Classify each day**:
   - **DayType** combines the company day-offs and the holiday calendar into Weekday, DayOff, Holiday or HolidayOnDayOff.
   - **Tags** depend on the row's *Rule Applied* column. With **External**, the values in the file are used (status, late, early, grace, extra hours). With **SystemSetting**, the system works the tags out from the punches and the notes.
   - On off-days, only the calendar tag and extra hours apply. Late, early, grace and Present are never set on an off-day.
4. **Fill the gaps**: a date missing from the file becomes `AttendanceMissing`. An enrolled employee who is not in the file gets `AttendanceUnavailable` for every day.
5. Import problems are written to `ERP.Attendance.Log.Errors` and shown as *Issues*.

Tag group	Tags (stored as numbers in `AttendanceTags`)
Data issues	AttendanceUnavailable 1, AttendanceMissing 2, AttendanceIncomplete 3, InvalidData 4
Presence and calendar	Present 5, DayOff 6, Holiday 7, HolidayOnDayOff 8, Absent 9, NotCounted 10
Timing	LateArrival 11, EarlyDeparture 12, GraceApplied 13

Leave (from keywords in the notes)	PaidLeave 14, UnpaidLeave 15
Extra hours	DayOffOvertime 16, HolidayOvertime 17, WeekdayOvertime 18
Manual	RegularizeIncomplete 19, RegularizeAbsent 20, RegularizeShortfall 21, ManualOverride 22

Mongo example – L3 daily row (ERP.Attendance.Log.DailyProcessedL3)

```
{
  _id: UUID("c3c3…18"), CompanyId: UUID("2222…"),
  DailyKey: "Mar16_2026_to_Apr15_2026_EMP-0001_20260318",
  SessionKey: "Mar16_2026_to_Apr15_2026", SessionUserKey: "Mar16_2026_to_Apr15_2026_EMP-0001",
  UserId: "EMP-0001", EmployeeId: UUID("3333…01"), EmployeeName: { en: "Sample Employee A" },
  WorkDate: ISODate("2026-03-18"),
  Status: 3,                          // 1 Imported, 3 Processed, 4 ModifiedAfterProcessed, 5 Published, 6 ModifiedAfterPublished, 7 Settled
  DayType: 1,                         // 1 Weekday, 2 DayOff, 3 Holiday, 4 HolidayOnDayOff
  AttendanceTags: [5, 11, 18],        // Present, LateArrival, WeekdayOvertime
  HolidayDetails: null, LeaveDetails: null,
  Record: {
    ShiftStart: "08:00:00", ShiftEnd: "17:00:00", RequiredHours: "8:00",
    EntryTime: "08:15:00", ExitTime: "17:30:00", HoursWorked: "9:15",
    LateCheckin: "0:15", EarlyCheckout: null, TotalShortfall: "0:15", DeductionHours: "0:15",
    GraceApplied: false,
    NormalExtraHours: "1:15", WeekendExtraHours: null, HolidayExtraHours: null, TotalExtraHours: "1:15",
    RuleApplied: 1                    // 1 External, 2 SystemSetting
  },
  AuditLog: [ { SerialNumber: 1, Action: 1, Message: "Imported from source", ModifiedBy: "system" } ]
}
// durations are stored as TimeSpan ticks; shown here as H:mm for readability
```

8. L4 processing, edits and publish

8.1 Finalise (L3 to L4)

Start-process (for the whole session or one employee) adds up each employee's L3 rows into one **L4 monthly document** per employee per session. It totals hours worked, late, early, shortfall, deduction and extra hours (normal, weekend and holiday), and counts days (present, absent, holiday, weekend, late, early departure, paid and unpaid leave). The session then moves to *Finalised*, and **Publish** locks it for payroll. Progress is saved on the session, so the UI can poll it.

Mongo example – L4 monthly (ERP.Attendance.Log.MonthlyLogL4, trimmed)

```
{
  Key: "Mar16_2026_to_Apr15_2026_EMP-0001", SessionKey:
"Mar16_2026_to_Apr15_2026",
  SessionId: UUID("5555…01"), UserId: "EMP-0001",
  PeriodStart: ISODate("2026-03-16"), PeriodEnd: ISODate("2026-04-
15"),
  Aggregate: {
    EmployeeCode: "EMP-0001", EmployeeName: "Sample Employee A",
    TotalMonthlyHoursWorked: "175:00", TotalMonthlyLateCheckIn:
"0:55", TotalMonthlyEarlyCheckout: "0:00",
    TotalMonthlyShortfall: "0:55", TotalMonthlyDeduction: "0:55",
    TotalMonthlyNormalExtraHours: "6:00",
TotalMonthlyWeekendExtraHours: "4:00",
    TotalMonthlyHolidayExtraHours: "0:00", TotalMonthlyExtraHours:
"10:00",
    TotalDaysInMonth: 31, TotalAbsentDays: 1, TotalHolidayDays: 1,
TotalWeekendDays: 8,
    TotalLateDays: 3, TotalEarlyDepartureDays: 0,
TotalPaidLeaveDays: 1, TotalUnpaidLeaveDays: 0,
    DayTypeCounts: { Weekday: 22, DayOff: 8, Holiday: 1 },
    EnrollmentRef: { EmployeeId: UUID("3333…01"), PayGroupId:
UUID("6666…01") }
  },
  Status: 1                          // AggregateStatus.Fresh
}
```

8.2 Editing L3 days

HR can correct a single day on the session grid. Every edit is saved in the row's `AuditLog` with a reason, and it marks the row and its L4 as modified (*ModifiedAfterProcessed* or *ModifiedAfterPublished*). **Re-process** then rebuilds only the employees whose rows changed. A row claimed by a payrun (*Settled*) cannot be edited.

Action	What it does
Edit / Override check-in-out	Changes the punches. Late, early, shortfall, deduction and extra hours are recalculated against the row's own shift, and the extra hours go to the bucket that matches the DayType. Adds the ManualOverride tag.
Regularize	Allowed only when the day is Absent or has a deduction. Recalculates the day against the row's shift and tags it RegularizeAbsent or RegularizeShortfall (plus Present on a weekday).
Mark Day Off / Mark Holiday	Reclassifies the day. Any worked time moves to weekend or holiday extra hours, and late, early, shortfall and deduction are cleared.
Override overtime	Sets total extra hours only.

Override deduction	Sets deduction hours only.
Clear	Reverts the last edit (one step back only).
Settle / Unsettle	Locks or unlocks days for an advance payrun.

9. Points to note

- An employee's location and holiday calendar come only from their work assignment map. If there is no map for the date, the API returns an error (2207) and does not fall back to another source.
- L3 values come from the file today. The import does not yet read assignment maps or policies, and calculating values from policies (SystemDefined mode) is planned. Grace is not yet applied to deductions, so deduction equals shortfall for now.
- At finalise, the L4 day counts are built from tags that the system works out again, not from the stored L3 tags, and the holiday check uses only the company default calendar for the session's start year. Both points need review.
- L5 (the payroll summary) still exists in the code, but it is not used. Reports and APIs read only up to L4.